ARCHITECTURE DECISION RECORDS

ADR-1: CONCURRENCY CONTROL FOR INVENTORY

Context: 10k concurrent requests, 100 units, hot single row.

Options considered:

1. Pessimistic (SELECT ... FOR UPDATE)
   Pros : Simple, correct
   Cons : Lock queue on one row; 10k waiting connections; low throughput; timeouts

2. Optimistic (version check + retry)
   Pros : No locks
   Cons : Under heavy contention ~all but one fail per round; retry storms

3. Atomic conditional UPDATE in DB
   Pros : Correct, simple, no explicit lock
   Cons : Still 10k writers hit one row

4. Redis Lua atomic gate + DB conditional UPDATE (CHOSEN)
   Pros : O(1) in-memory admission; only 100 reach DB; DB constraints as safety net
   Cons : Two stores; needs reconciliation and rebuild procedure

Decision   : Redis gate for admission, Postgres conditional update as authority.
Consequences: Reconciler, rebuild-from-DB runbook, and drift alerts required.

ADR-2: SQL vs NoSQL

Decision : PostgreSQL for inventory, payments, orders (ACID, constraints, unique keys, CHECK invariants). NoSQL (e.g. Cassandra/Dynamo) offers write scale but weak multi-row invariants; Redis used only as cache/gate, never as the record of truth. Catalogue/search may use a document store/cache.
Trade-off : Vertical limits on one primary; mitigated by funnel design and per-service DBs.

ADR-3: SYNC vs ASYNC

Decision : Sync for reserve and payment initiation (user needs immediate outcome). Async (Kafka + outbox) for order creation, inventory finalisation, shipment, notification.
Trade-off : Eventual consistency and extra moving parts, in exchange for isolation from downstream outages.

ADR-4: PAYMENT IDEMPOTENCY AND UNKNOWN OUTCOMES

Decision : One payment per reservation (unique key); gateway idempotency key = payment_id; timeouts yield UNKNOWN resolved by status query, never by new charge.
Trade-off : User waits longer on ambiguous cases.

ADR-5: CACHE STRATEGY

Decision : CDN + cache-aside Redis for catalogue (TTL 5-30 s, coalescing);

```
            stock DISPLAY is eventually consistent via edge flag; stock
            DECISION is never taken from cache.
Trade-off : User may briefly see "available" then get SOLD_OUT.


ADR-6: RESERVATION EXPIRY
-------------------------
Decision  : Redis TTL on gate keys + DB expires_at sweeper (SKIP LOCKED), DB
            is authoritative. Payment in flight extends expiry once by
            grace period.
Trade-off : Units held up to TTL + grace by unpaid users.


ADR-7: ORCHESTRATION vs CHOREOGRAPHY
------------------------------------
Decision  : Choreography via events for post-payment flow; Checkout Facade
            orchestrates only the synchronous pay step. Reconciler provides
            the safety net a saga coordinator would.
Trade-off : Flow is spread across services; mitigated by tracing.


ADR-8: SINGLE-REGION BUY PATH
-----------------------------
Decision  : One inventory authority in one region (multi-AZ).
Trade-off : Region outage stops sales; accepted for correctness and
            simplicity.
```